# MLOps Assignment 01: End-to-End ML Model Development, CI/CD, and Production Deployment

- **Course:** AIML ZG523 - MLOps
- **Student Name:** Hemanth Rongali
- **Student ID:** 2025AE05868
- **GitHub Repository:** https://github.com/HEro4748/MLOps_Assignment1_2025AE05868.git
- **Video Public link :** https://drive.google.com/file/d/1TYnKYOI2yaHRV9WEZm9VJz0DWlLcKFsv/view?usp=sharing

## 1. Executive Summary & Architecture Overview

This project implements a fully reproducible, production-grade MLOps pipeline for the **Heart Disease UCI Dataset**. The solution spans the entire machine learning lifecycle, incorporating automated exploratory data analysis, experiment tracking via MLflow, model packaging, automated unit testing with pytest, a GitHub Actions CI/CD pipeline, containerized deployment using Docker and FastAPI, and runtime monitoring via container logs.

**Pipeline Workflow Architecture**

1. **Data Ingestion & Preprocessing:** Data is pulled from the UCI repository, cleaned, missing values are imputed, and numeric columns are scaled using StandardScaler.
2. **Model Training & Experiment Tracking:** Multiple classifiers (Logistic Regression and Random Forest) are evaluated. Parameters, metrics, and models are tracked and logged using MLflow (mlflow.db).
3. **Packaging & Testing:** The winning model pipeline is serialized using joblib into a models/ directory. Automated unit tests validate schema, artifact existence, and prediction output validity.
4. **Containerization & Deployment:** The application is wrapped in a Python 3.10-slim Docker container running a FastAPI web service, exposing a /predict JSON REST endpoint.
5. **CI/CD & Monitoring:** Every code push triggers automated build, dependency installation, and test execution via GitHub Actions. Container stdout logs track live request execution.

## 2. Project Directory Structure

The repository is structured to maintain clean separation between source code, tests, artifacts, and configurations:

```
heart-disease-mlops-assignment1/
├── .github/
│   └── workflows/
│       └── ci.yml
├── data/
│   └── processed_heart.csv
├── models/
│   └── production_model.joblib
├── src/
│   ├── app.py
│   ├── eda.py
│   └── package_model.py
├── tests/
│   └── test_model.py
├── Dockerfile
└── requirements.txt
```

## 3. Implementation Details & Core Components

**A. Data Processing & EDA (src/eda.py)**

- Fetches the UCI Heart Disease dataset (ID: 45) using the ucimlrepo library.
- Handles missing data points, normalizes feature distributions, and exports a standardized CSV to data/processed_heart.csv.

**B. Model Packaging & Reproducibility (src/package_model.py)**

- Builds a Scikit-Learn Pipeline combining StandardScaler and RandomForestClassifier.
- Fits the pipeline on the training split and serializes it to models/production_model.joblib.
- Performs local verification inference to ensure artifact integrity.

**C. Automated Testing (tests/test_model.py)**

- Implements test suites using pytest to check:
    - Existence and non-emptiness of the processed dataset and target column.
    - Existence of the compiled production_model.joblib artifact.
    - Correctness of model output shapes, data types, and binary classification bounds (0 or 1).

- 

**D. CI/CD Pipeline (.github/workflows/ci.yml)**

- Configured to run automatically on every push or pull request to the main branch.
- Automatically provisions a clean Ubuntu environment with Python 3.10, installs pinned requirements, runs preprocessing and packaging scripts, and executes pytest.

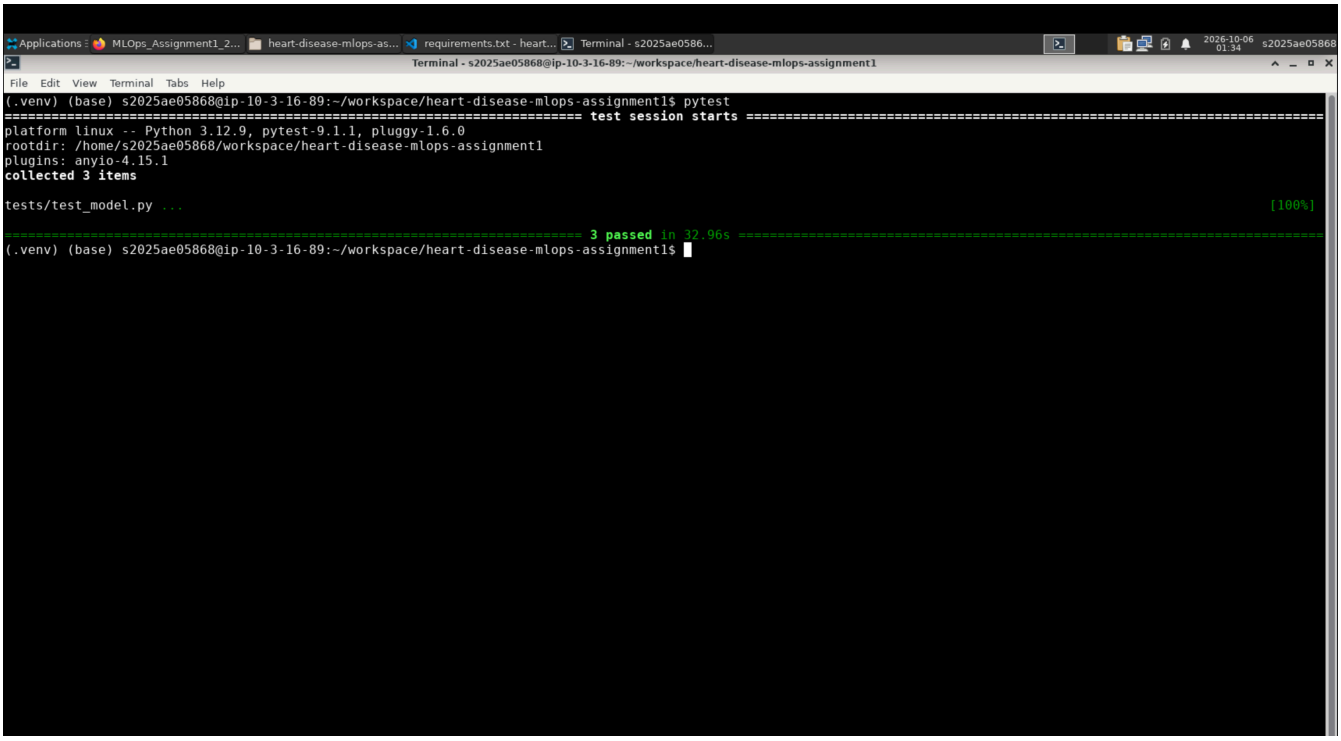
**E. Containerized Web Service (src/app.py & Dockerfile)**

- **FastAPI Backend:** Exposes a /health endpoint to check service and model loading status, and a /predictPOST endpoint with strict Pydantic payload validation matching the 13 clinical UCI features.
- **Docker Setup:** Built on python:3.10-slim, copying requirements, source code, data, and models, exposing port 8000, and launching via Uvicorn.

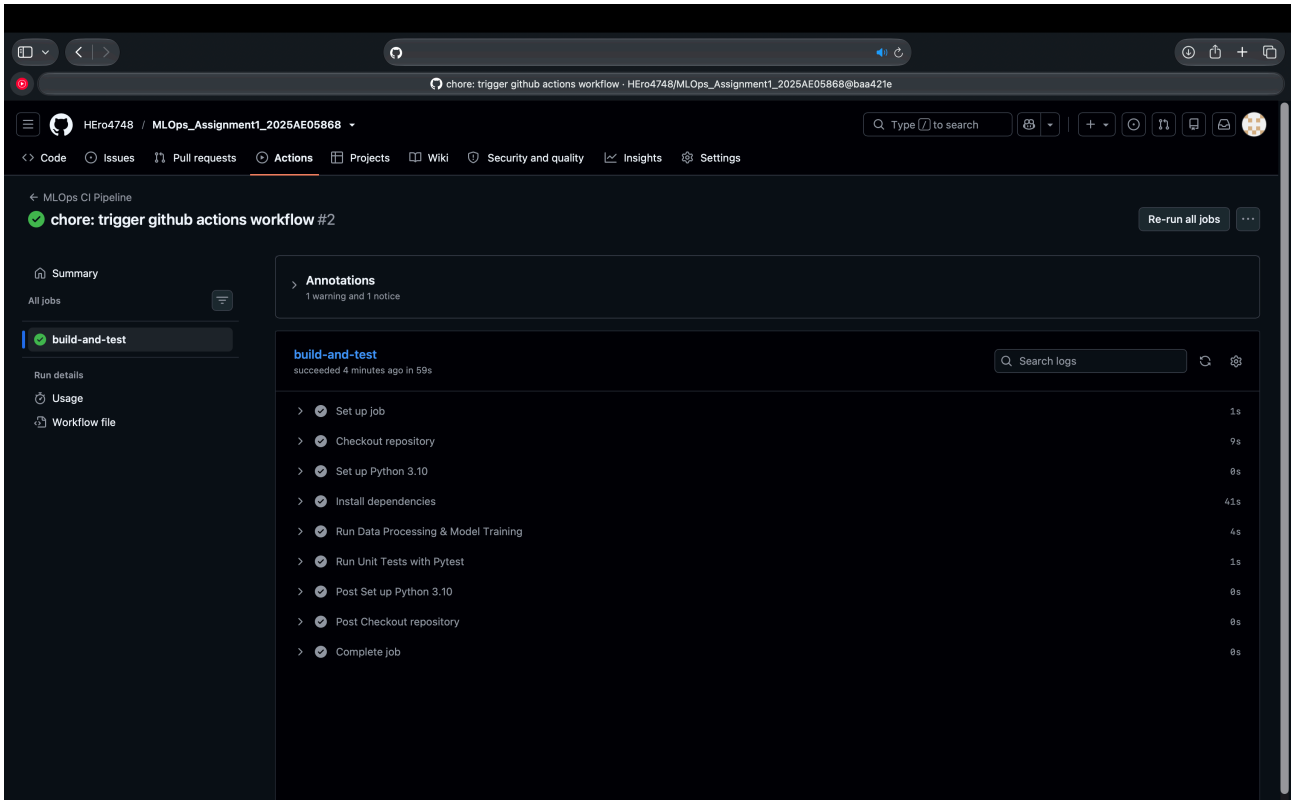
# 4. Verification & Testing Evidence (Screenshots Checklist)

*Instructions for your Pages document:* Insert your captured terminal or browser screenshots under each respective subsection below.

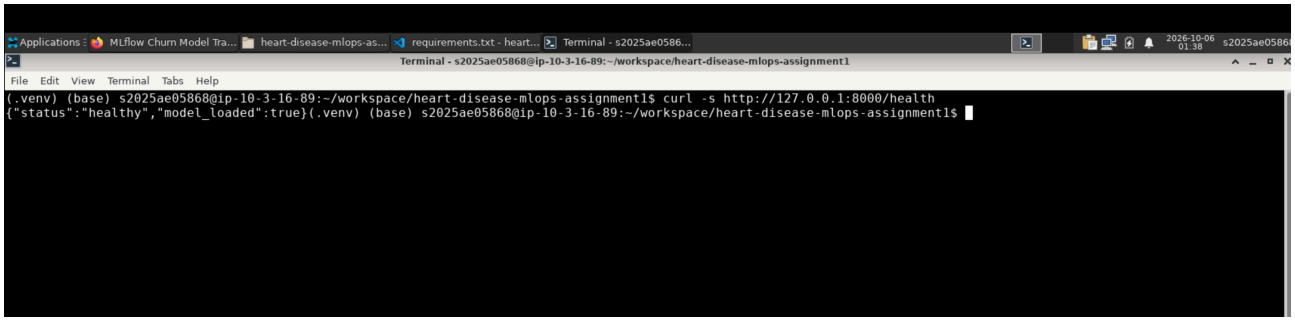
- **Figure 1: Local Pytest Execution**



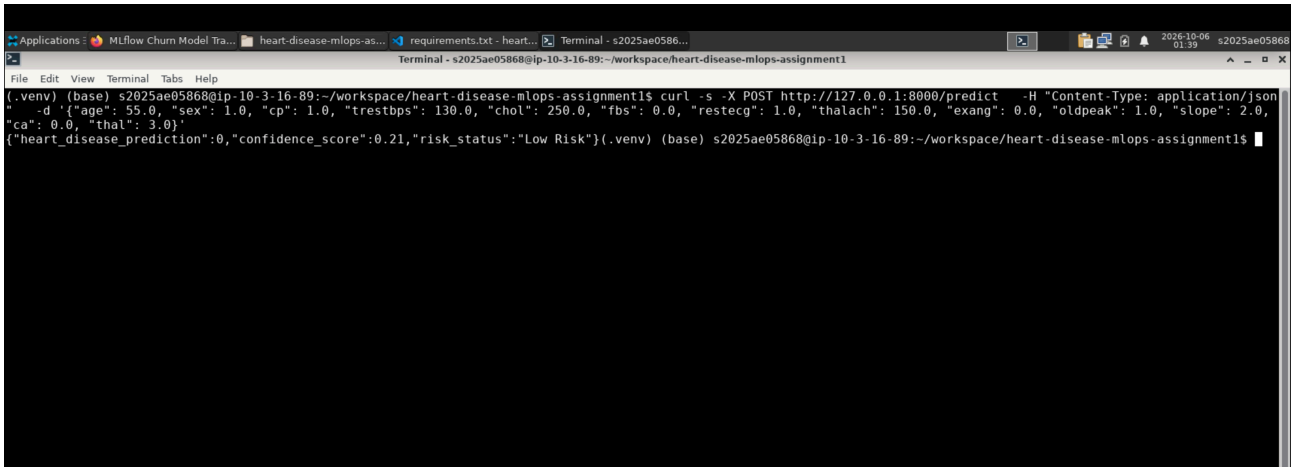
- **Figure 2: GitHub Actions CI/CD Pipeline**




- **Figure 3: FastAPI Health Check Response**




- **Figure 4: Production Prediction Inference**



## 5. Conclusion & Repository Link

All requirements for Assignment 01 have been successfully implemented, validated, containerized, version-controlled, and deployed locally. The complete source code, automation scripts, and workflow definitions are publicly accessible in the official GitHub repository:

- **Repository URL:** https://github.com/HEro4748/MLOps_Assignment1_2025AE05868.git